

Smart Home IoT System

Software Engineering for the Internet of Things

Project Report

Name & Surname	Matriculation	E-mail address
Agostino D'Agostino	303226	Agostino.DAgostino@student.univaq.it
Alessandro Di Giacomo	303377	Alessandro.DiGiacomo@student.univaq.it
Graziano D'Agostino	311416	Graziano.dagostino@student.univaq.it

Università degli Studi dell'Aquila

June 22, 2026

Contents

Objective	2
Architecture Overview	3
Applicable Domains	5
Functional Requirements	6
1. Sensor Integration	6
2. Communication Protocols	8
3. Data Processing	9
4. Data Storage	10
5. Visualization	11
6. Alerting Mechanisms	12
Testing	15
Non-Functional Requirements	19

Objective

The project requires a complete IoT system that collects, processes, stores, and visualises data from different domains. The system should show key IoT concepts and work across multiple application areas. We built a Smart Home implementation that goes from sensor data acquisition to visualisation and automated actuation, with all components containerised for portable deployment.

The six points of the project scope are addressed as follows. Sensors are simulated in software for data collection. Node-RED is the middleware processing layer. InfluxDB provides time-series storage. Grafana renders an interactive dashboard. Configurable thresholds drive Telegram-based alerting. Every component runs as a Docker container orchestrated through Docker Compose.

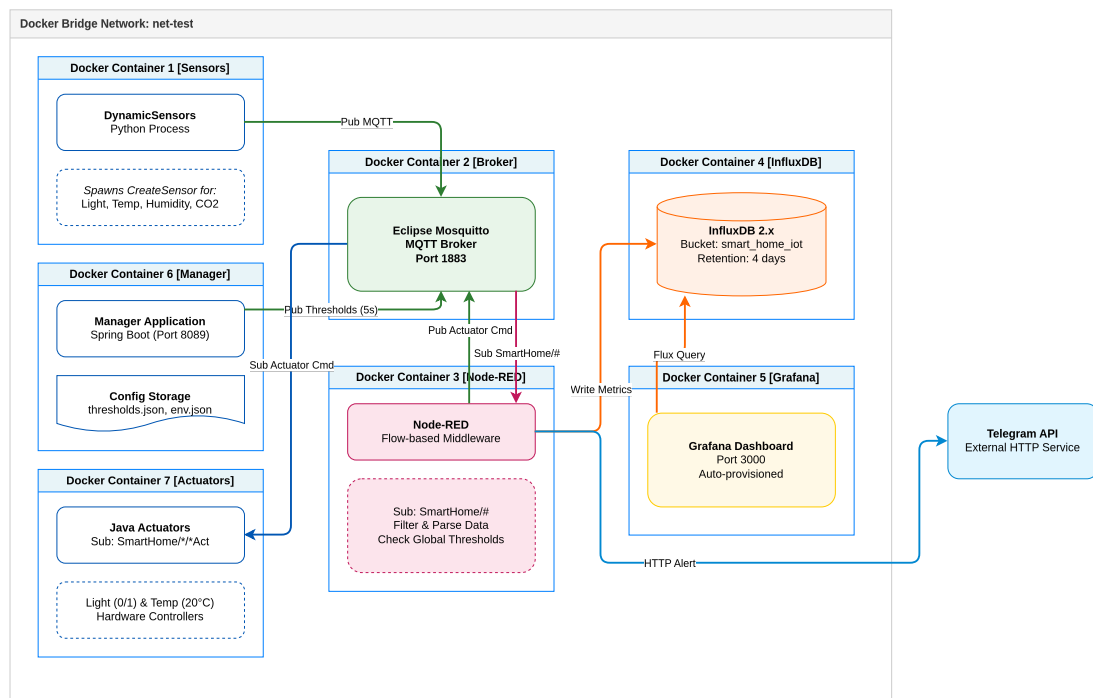


Figure 1: System architecture: MQTT communication between sensors, broker, and applications in a containerised environment.

Architecture Overview

The system architecture is organised in two complementary views, as shown in Figure 2. The upper part illustrates the Docker-based deployment: seven containers (Sensors, Broker, Node-RED, InfluxDB, Grafana, Manager, Actuators) communicate over a shared bridge network via MQTT, HTTP, and InfluxDB protocols. The lower part abstracts the same system into four logical layers Device & Data, Communication, Middleware & Storage, and Application highlighting how data flows from physical sensors through the MQTT broker and processing pipeline up to the visualisation and management dashboards.

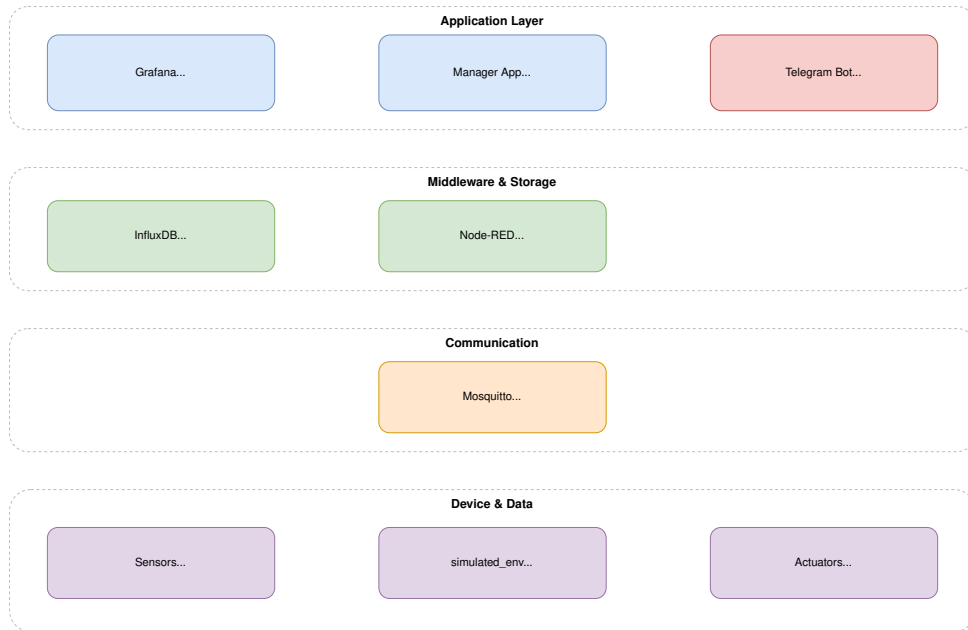


Figure 2: System architecture: Docker deployment view (top) and logical layered view (bottom).

This layered structure means each component can evolve independently. Adding a new sensor type does not require changes to Node-RED or Grafana, and replacing the visualisation tool would not affect data collection.

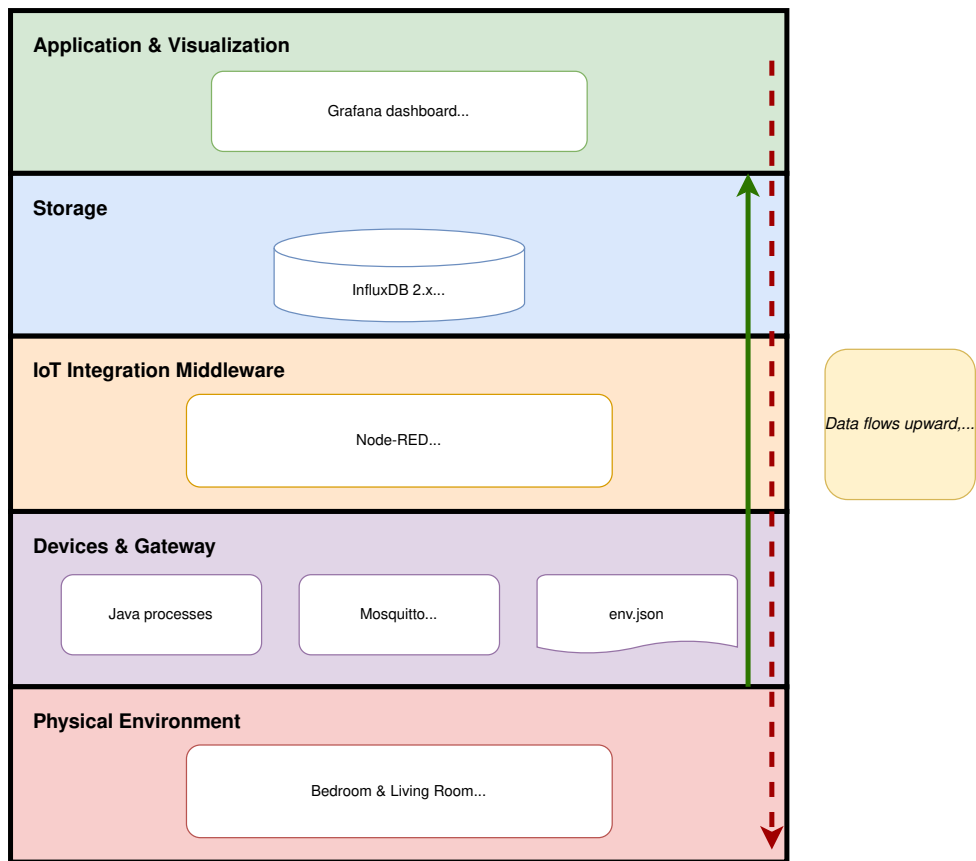


Figure 3: Layered block architecture diagram.

Applicable Domains

The requirement document lists six possible application domains: Smart Agriculture, Smart Cities, Industrial IoT, Environmental Monitoring, Healthcare, and Smart Homes. We chose the **Smart Home** domain. Two rooms (bedroom and living room) are monitored for light intensity, temperature, humidity, and CO₂ concentration. Actuators for lighting and temperature set-points provide automatic responses to threshold violations.

We picked Smart Home because it is familiar and involves multiple sensor types: scalar measurements (temperature, light), percentage-based readings (humidity), and binary or set-point actuation (light on/off, temperature target).

Functional Requirements

1. Sensor Integration

The requirements ask for simulated or real sensors that publish data periodically with realistic ranges, timestamps, and unique identifiers.

We built the sensor subsystem in Java 17 with two classes. `DynamicSensors.java` is the entry point. It polls the shared environment file `simulated_env/env.json` every five seconds and starts a `CreateSensor` process for each sensor entry it finds. Sensors can be added or removed at runtime without restarting anything.

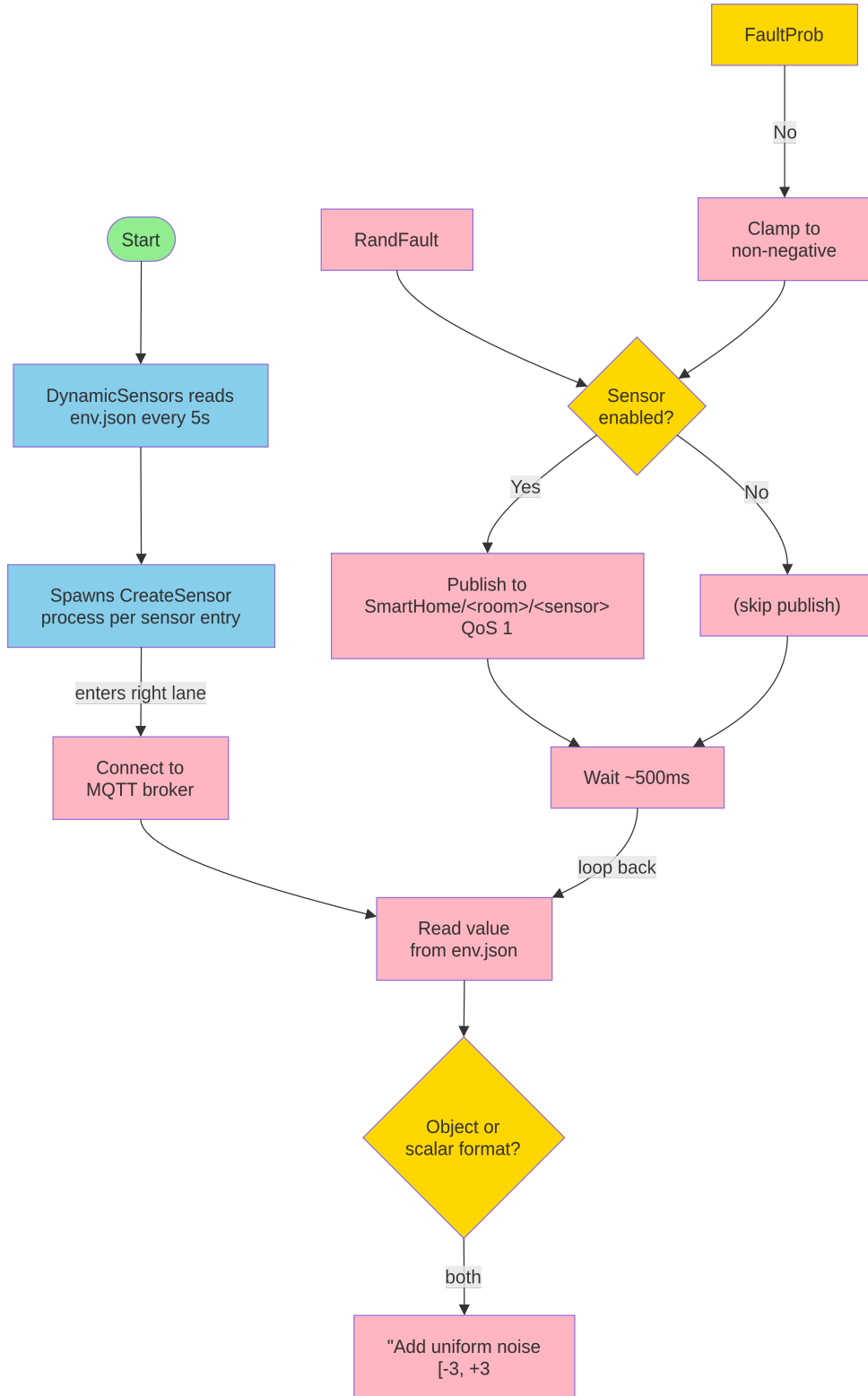


Figure 4: Sensor operation cycle.

Each `CreateSensor` instance connects to the MQTT broker and enters a publishing loop. On each iteration it reads the sensor value from `env.json`. The code handles both scalar values (e.g., `"light": 0`) and structured objects with an `enabled` flag (e.g., `"humidity": {"value": 30, "enabled": true}`). It adds uniform noise drawn from $\{-3, -2, -1, 0, 1, 2, 3\}$ to the base value and clamps the result to non-negative

values. With 1% probability the value is replaced with a random integer between 6 and 10 to simulate faults. The result is published as a string to `SmartHome/<room>/<sensor>` with QoS 1, roughly every 500 milliseconds. If a sensor is disabled, the process skips publishing.

Four sensor types are deployed across the two rooms: light, temperature, humidity, and CO₂. Each topic combines the room name and sensor type, so every data stream has a unique identifier.

2. Communication Protocols

The requirements specify MQTT as the primary protocol, with well-structured topics, a broker, and support for dynamic configuration.

Why MQTT. The course compares several IoT protocols: MQTT, CoAP, HTTP, AMQP, and DDS. We chose MQTT because it fits the project’s needs better than the alternatives. CoAP uses a request-response model over UDP and is designed for extremely constrained devices, but our sensors run on Java with full network stacks where CoAP’s lightweight advantages are less relevant. HTTP, while ubiquitous, follows a pull model that requires polling and generates significantly more bytes on the wire a drawback for frequent sensor updates. MQTT’s publish-subscribe pattern matches the event-driven nature of sensor data: readings arrive asynchronously and must be pushed to multiple consumers (storage, alerting, actuation) without requiring each consumer to poll. The oneM2M standard, discussed during the course, defines MQTT as a recognised protocol binding for IoT communication, further motivating its adoption.

QoS choices. MQTT provides three Quality of Service levels. Sensors publish with QoS 1 (at least once) to ensure readings are not lost in transit due to network interruptions, at the cost of possible duplicates. Node-RED subscribes with QoS 2 (exactly once) for the threshold and alerting path, where duplicate messages could cause false alarms. The broker (Eclipse Mosquitto) handles the retry logic automatically, storing undelivered messages for disconnected subscribers.

MQTT broker. Eclipse Mosquitto runs in its own Docker container on port 1883. In this development setup the broker allows anonymous connections. Authentication and TLS would be needed for production.

Topic hierarchy. We adapted the suggested pattern `/<domain>/<location>/<sensor>` to the Smart Home domain:

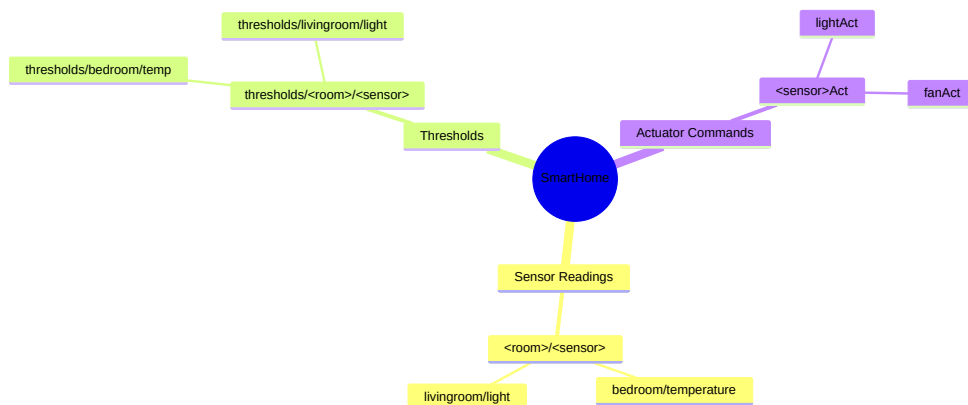


Figure 5: MQTT topic hierarchy.

```

1 SmartHome/<room>/<sensor>                # sensor readings
2 SmartHome/thresholds/<room>/<sensor>      # threshold updates
3 SmartHome/<room>/<sensor>Act               # actuator commands

```

Listing 1: Struttura dei topic MQTT

Concrete examples are:

- SmartHome/bedroom/temperature
- SmartHome/thresholds/livingroom/light
- SmartHome/bedroom/lightAct

This three-family structure separates sensor data, configuration, and commands into independent streams. Subscribers use MQTT wildcards to pick the category they need. Node-RED subscribes to `SmartHome/#` for everything, while an individual actuator subscribes only to its specific `Act` topic.

Dynamic configuration. The Manager Application publishes threshold updates to `SmartHome/thresholds/<room>/<sensor>` every five seconds. Node-RED picks up these messages and updates its in-memory thresholds on the fly. No restart needed when thresholds change.

3. Data Processing

The requirements ask for a middleware layer that aggregates, transforms, and enriches sensor data before sending it to storage and visualisation. Node-RED handles this with its flow-based programming model.

Batch vs stream processing. The course distinguishes two processing paradigms. Batch processing operates on stored data in fixed time intervals and is suited for offline analytics. Stream processing, on the other hand, handles each data point as it arrives, making it appropriate for real-time monitoring and alerting. Our system is a stream processing architecture: every MQTT message is processed immediately upon arrival, and the pipeline produces alerts, database writes, and actuator commands within milliseconds of a sensor reading. There is no polling or periodic batch job in the critical path. This approach follows the general data pipeline workflow described in the course (Data Generation → Transformation → Ingestion → DB → Processing), with the difference that transformation and processing happen concurrently in Node-RED rather than in separate stages.

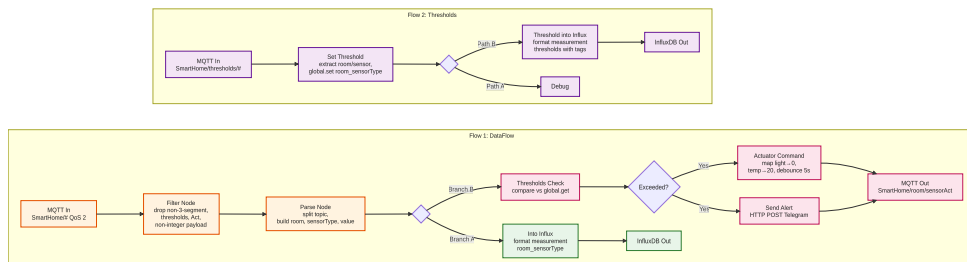


Figure 6: Node-RED processing pipeline.

The processing pipeline has two flows.

DataFlow tab. The main pipeline starts with an MQTT input node subscribed to `SmartHome/#` with QoS 2. A filter node drops any message whose topic does not have exactly three segments, contains `thresholds`, ends in `Act`, or has a non-integer payload. This keeps only clean sensor readings. A parsing node splits the topic on `/` and builds a structured message with `room`, `sensorType`, and `value`.

From here the flow forks into parallel branches. One branch writes the measurement to InfluxDB under `{room}_{sensorType}` with tags `room` and `sensorType`. Another branch compares the value against the threshold from Node-RED's global context using the key `{room}_{sensorType}`. If the threshold is exceeded, the message goes to the alerting and actuation sub-flows.

Thresholds tab. A second flow subscribes to `SmartHome/thresholds/#`, extracts room and sensor type from the topic, updates the corresponding entry in Node-RED's global context, and writes the change to InfluxDB for auditing.

4. Data Storage

The requirements specify a time-series database with data tagged by timestamp, sensor type, location, and domain.

Why a time-series database. Sensor data has specific characteristics that make relational databases a poor fit. Readings are immutable and append-only, queries scan contiguous time ranges rather than random rows, and the write volume is high (multiple sensors publishing every 500 ms). A relational database like PostgreSQL can store this data, but the schema needs constant updates when sensors are added, write performance degrades as the table grows, and horizontal scaling is complex. Time-series databases are built for these workloads: they use columnar compression, partition data by time for efficient range scans, and support retention policies that expire old data automatically. The course presents InfluxDB, Prometheus, and TimescaleDB as options; we chose InfluxDB 2.x for its integrated query language (Flux), its easy Docker setup, and its compatibility with Grafana.

We used **InfluxDB 2.x** with a single bucket called `smart_home_iot` under organisation `univaq`. The retention period is four days.

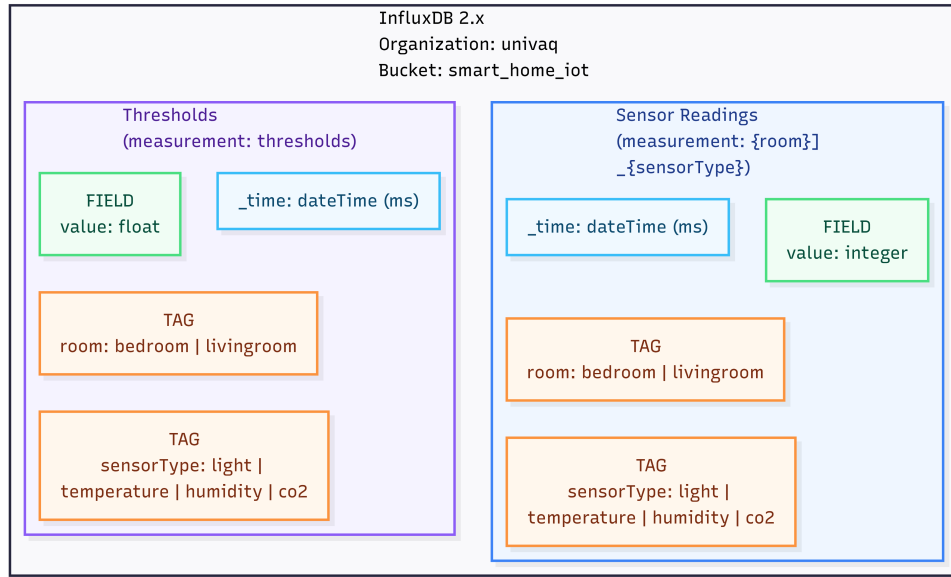


Figure 7: InfluxDB data model schema.

Sensor readings go into measurements named `{room}_{sensorType}` (e.g., `bedroom_light`, `livingroom_temperature`). Each point has two tags (`room` and `sensorType`) and one integer field (`value`) with millisecond-precision timestamps. Threshold changes are stored in a separate `thresholds` measurement with the same tag structure but a float `value`. Anyone reading the data can reconstruct the full context from the measurement name and tags alone.

The database initialises automatically at container startup through `DOCKER_INFLUXDB_INIT_*` environment variables that set the admin credentials, organisation, bucket, and retention policy.

5. Visualization

The requirements ask for an interactive dashboard with real-time graphs, gauges, filtering, and visual highlighting of threshold violations.

Grafana provisions itself automatically at startup from configuration files mounted as Docker volumes. A YAML datasource definition points to InfluxDB with the URL, organisation, bucket, and auth token. A JSON dashboard definition creates the Smart Home IoT dashboard (uid `smarthome-main`).

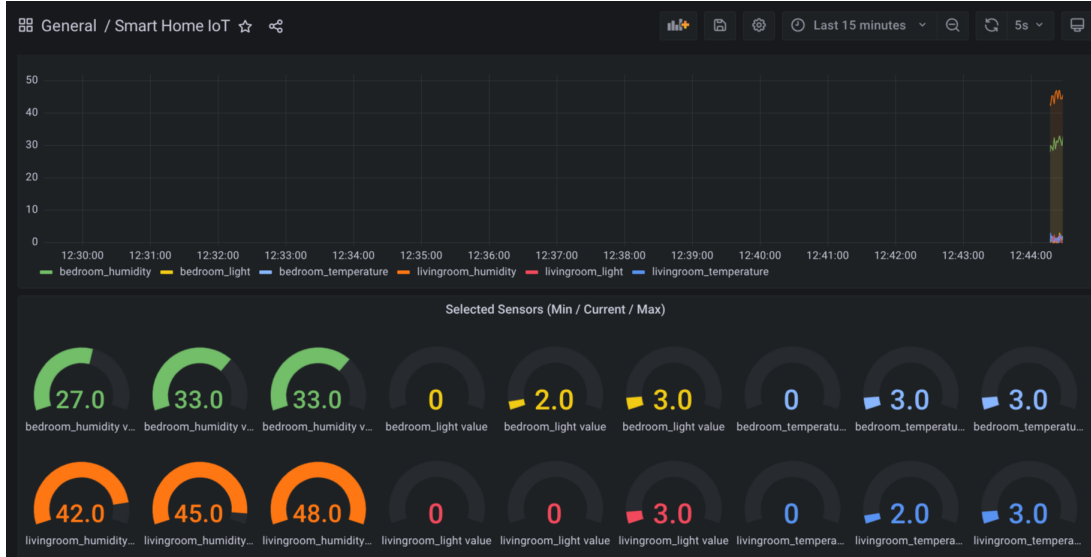


Figure 8: Grafana dashboard screenshot.

The dashboard has two panels. A time-series line chart shows sensor values over a configurable time window (default 15 minutes) and refreshes every five seconds. A gauge panel shows minimum, current, and maximum values for selected sensors, with threshold violations in red. A template variable `$measurement` lets the user filter by measurement name.

The dashboard uses Flux, the InfluxDB query language. Flux’s pipeline syntax chains filtering, aggregation, and windowing as stream transformations, which fits the time-series nature of the data.

6. Alerting Mechanisms

The requirements ask for configurable thresholds and notifications through messaging platforms, with structured messages that identify the domain, sensor, location, and violated condition.

Threshold configuration. Default thresholds live in the shared file `simulated_env/thresholds.json`. The Manager Application provides a web interface (port 8089) to view and edit these values at runtime. You can add new sensors and enable or disable existing ones from the same interface.

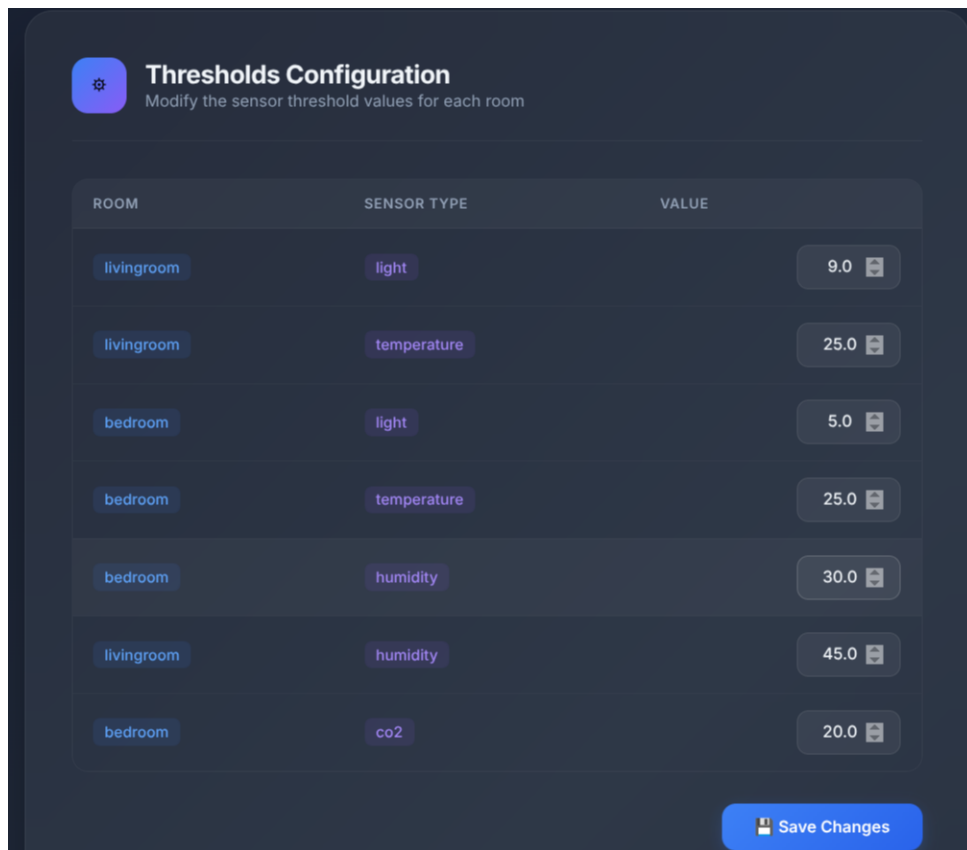


Figure 9: Manager Application threshold management page.

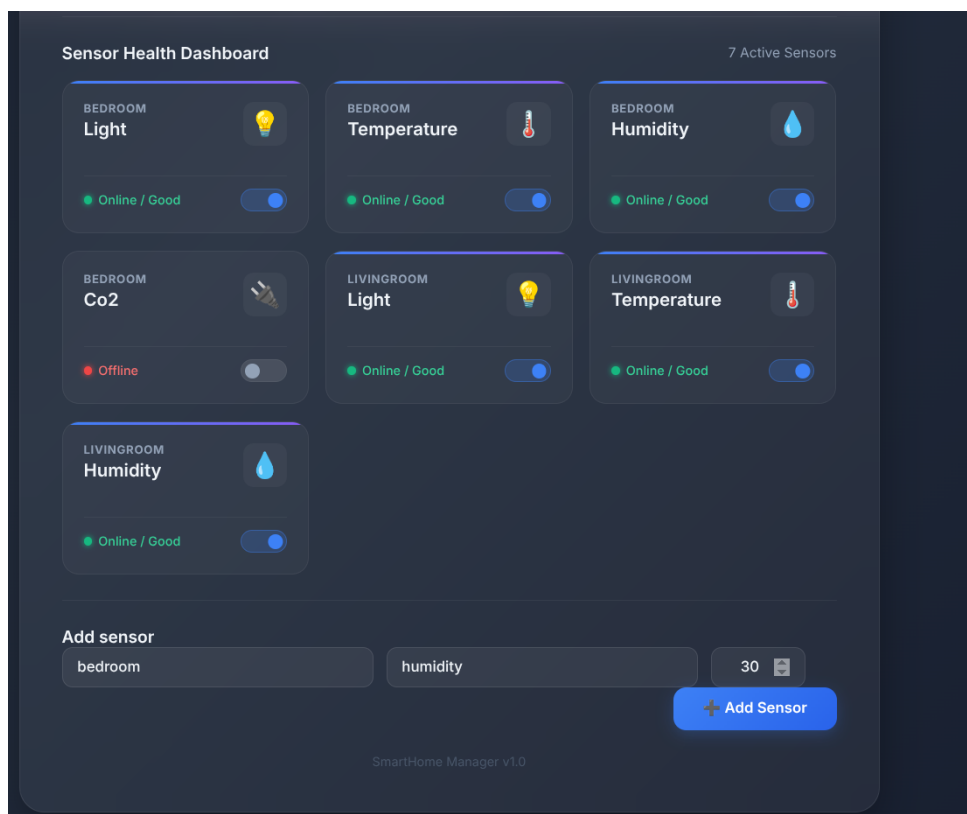


Figure 10: Manager Application sensor status and enablement page.

Alert trigger. Threshold checking happens in Node-RED's threshold check node. It retrieves the threshold for a sensor from the global context using the composite key `{room}_{sensorType}`. If the sensor value exceeds the threshold, the message goes to the alerting sub-flow.

Telegram notifications. The alert sub-flow creates an HTTP POST request to the Telegram Bot API at <https://api.telegram.org/bot<token>/sendMessage>. The message body uses the structured format from the specification:

```
1 Domain: SmartHome
2 Location: <room>
3 Sensor: <sensorType>
4 Condition: Exceeded max value
```

Listing 2: Alert message format

The bot token and chat ID come from the environment variables `TGBOT_TOKEN` and `TG_CHAT_ID`.

Actuator automation. When a threshold is breached, the system notifies and acts. For light sensors it publishes "0" (turn off) to `SmartHome/<room>/lightAct`. For temperature sensors it publishes "20" (set-point) to `SmartHome/<room>/temperatureAct`. A debounce limits actuation to one command per five seconds per sensor to avoid oscillation.

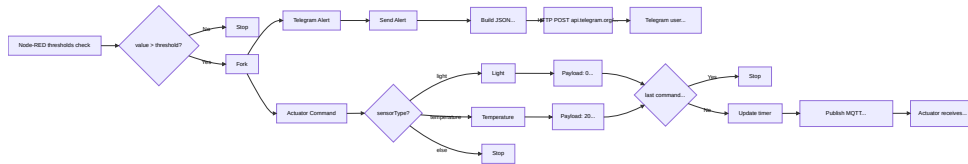


Figure 11: Alert and actuation flow diagram.

Testing

Testing an IoT system presents unique challenges. Unlike a standard web application, an IoT stack spans multiple technologies (MQTT, Node-RED, InfluxDB, Docker) and failure modes include network interruptions, malformed messages, file corruption, and misconfigured containers. We structured the tests into three levels: unit tests for the business logic (Manager Application), validation scripts for the infrastructure (Docker Compose), and validation scripts for the processing pipeline (Node-RED flows).

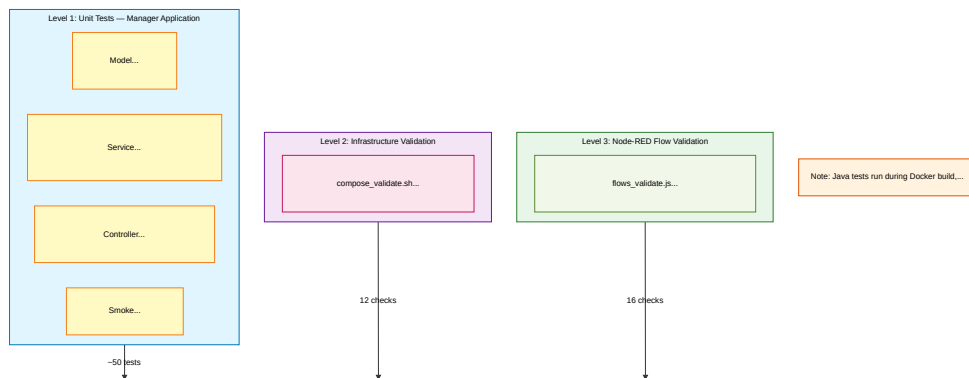


Figure 12: Schema of the three test levels and their coverage.

Unit Tests Manager Application

The Manager Application is the component with the most complex business logic: it reads and writes JSON files for thresholds and environment state, publishes MQTT messages, and serves a web UI. We used JUnit 5 with Mockito to test each layer in isolation.

Model layer

ThresholdTest verifies that the **Threshold** POJO correctly stores room, sensorType, and value. The `equals()` method is tested to confirm that two thresholds with the same room and sensorType are considered equal regardless of their valuea deliberate design choice since the form submission uses `equals()` to identify which thresholds to update.

SensorTest tests the **Sensor** POJO, including the health status logic: an enabled sensor has “Good” health, while a disabled sensor is marked “Offline”. This mapping is used by the web UI to show status indicators.

ThresholdsFormTest tests the wrapper class used by Thymeleaf to bind form data. It ensures the form works with an empty list, which can happen when the thresholds file is missing or empty.

Service layer

ThresholdsServiceImplTest is the most comprehensive test class. It uses `@TempDir` to create isolated JSON files and Mockito to simulate the MQTT client without requiring a real broker. The tests cover five areas.

`GetThresholds` validates reading from `thresholds.json`: a valid file, an empty array, and a missing file (which should throw `IOException`).

`UpdateThresholds` verifies that writing a list of thresholds produces correct JSON output.

`AddThreshold` tests the full flow of adding a new sensor: file update, duplicate detection (`IllegalArgumentException` for existing room+sensorType), automatic trimming and lowercasing of names, and synchronisation with `env.json`.

`GetSensors` parses the environment file in both formats the current structured format (`{value, enabled}`) and the legacy plain integer format and correctly produces `Sensor` objects with the right health status.

`ToggleSensorStatus` verifies enabling and disabling sensors, including the conversion of legacy plain-integer entries to the structured format when a sensor is toggled for the first time. Edge cases such as unknown rooms, unknown sensors, and missing files are handled without throwing exceptions.

`PublishThresholdsMQTT` tests the scheduled task that broadcasts thresholds every five seconds. It covers five scenarios: already connected (publishes directly), disconnected (connects then publishes), multiple thresholds (publishes to separate topics), connection failure (handles `MqttException` gracefully), and missing file (handles `IOException` gracefully).

Controller layer

ManagerControllerTest uses `@WebMvcTest` with `MockMvc` to test all four endpoints without starting a full HTTP server. The ten test methods cover the success path and the error path for each endpoint: `GET /` returns the thresholds page, `POST /changeThresholds` redirects with success or error messages, `POST /addSensor` handles success, duplicates, and internal errors, and `POST /sensorEnabled` correctly reports the new state (“On” or “Off”).

Smoke test

ManagerApplicationTest is a `@SpringBootTest` that loads the full Spring context. If any bean fails to instantiate or a dependency is missing, the test fails immediately. This catches configuration errors that unit tests cannot detect.

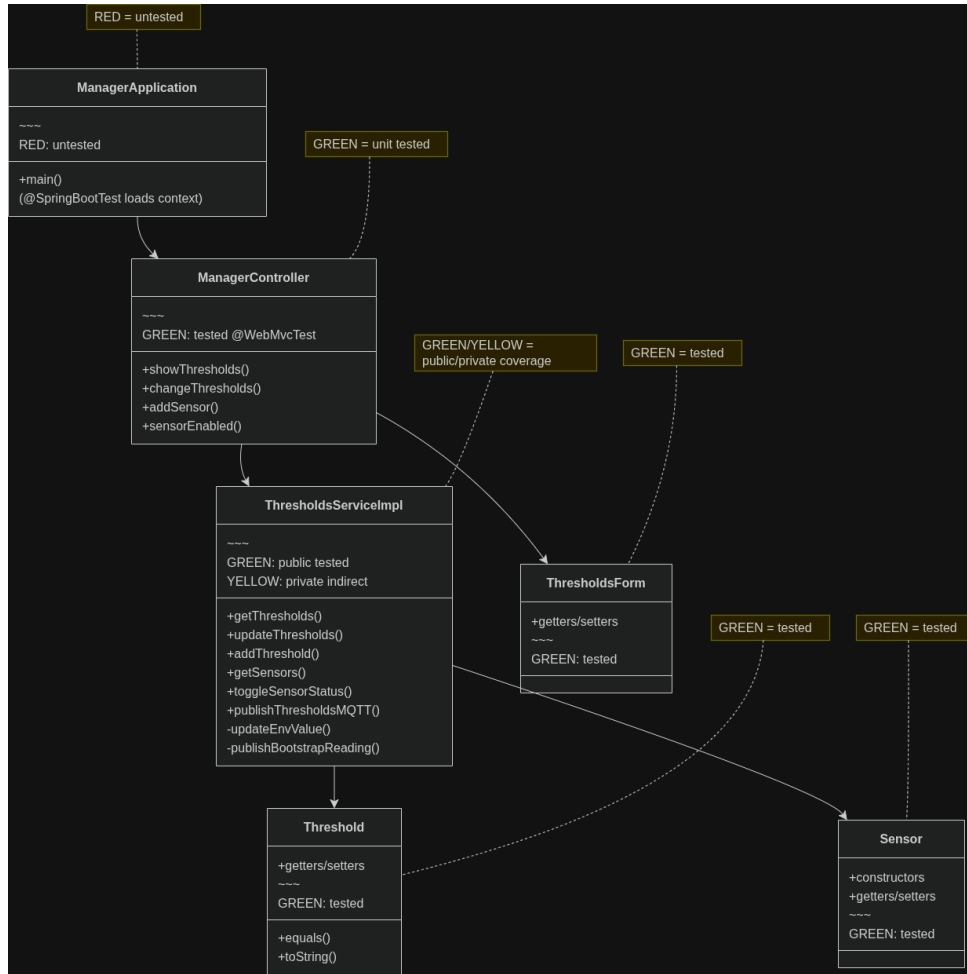


Figure 13: Manager Application test coverage diagram.

Infrastructure Validation

compose_validate.sh is a bash script that validates the Docker Compose setup. It checks that **compose.yaml** is syntactically valid, the **.env** file exists, all referenced Dockerfiles and configuration files are present, every service defines either an image or a build context, and every service is attached to the **net-test** network. Since an error in the compose file blocks the entire deployment, this script acts as a syntax and sanity gate before running **docker compose up**.

Node-RED Flow Validation

flows_validate.js validates the Node-RED **flows.json** file. The Node-RED flow contains the core processing logic: filtering, parsing, threshold checking, Telegram alerts, InfluxDB writes, and actuator commands. An invalid flow causes silent misbehaviour: alerts might not fire, data might not be stored, or actuators might not respond without any error log. The script checks that the JSON is well-formed, every node has an id and type, all wire connections point to existing nodes, the JavaScript code in every Function node is syntactically valid, MQTT nodes reference a configured broker, and InfluxDB nodes reference a valid configuration.

How Tests Are Run

```
1 # Unit test della Manager Application
2 cd managerApplication/app/managerApp && ./mvnw test
3
4 # Validazione Docker Compose
5 ./tests/compose_validate.sh
6
7 # Validazione Node-RED flows
8 node tests/flows_validate.js
```

Listing 3: Comandi per eseguire i test

The Java tests are also executed automatically during the Docker build, inside the multi-stage Dockerfile, before the application JAR is packaged. This prevents a container image from being built if the tests fail.

Non-Functional Requirements

Portability. Every component runs in a Docker container. The whole stack is defined in one `compose.yaml` file. All seven containers connect to a shared bridge network (`net-test`). Two named volumes provide persistent storage for InfluxDB and Grafana. The `simulated_env` directory is bind-mounted into the sensors, actuators, and manager containers for shared file access. One `docker compose up -d --build` command deploys everything on any machine with Docker.

Scalability. The MQTT topic structure separates data streams by room and sensor type. Adding a new sensor just means publishing to a new topic. No changes to the broker, Node-RED flows, or database schema are needed. The Manager Application has an `addSensor` endpoint to register the new sensor's threshold and environment entry from the web interface.

Resilience. Actuators retry MQTT connections up to ten times on connection loss. File operations on the shared environment use exclusive file locking (`FileLock`) and atomic renames. Writes go to a temporary file first, then get atomically moved to the target path. This prevents partial writes and race conditions when multiple actuators write at the same time.

User-Friendly Design. Grafana auto-provisions on startup with no manual setup. The Manager Application has a dark-themed web UI with sensor health indicators, an editable threshold table, and an add-sensor form. The project README provides quick-start instructions and documentation.

Security. InfluxDB requires authentication through an admin token configured at startup. Each component exposes only the ports it needs: Mosquitto on 1883, Node-RED on 1880, InfluxDB on 8086, Grafana on 3000, Manager App on 8089. The MQTT broker allows anonymous connections in this development setup. Adding username/password authentication or TLS would be the first step before moving to production.

Docker Services Summary				
Service	Image/Build	Port	Depends On	Volumes
broker	eclipse-mosquitto	1883	-	mosquitto.conf.ro
nodered	nodered/node-red:latest	1880	broker	./nodered:/data
influxdb	influxdb:2.1.1	8086	-	influxdb-storage:/var/lib/influxdb2
grafana	grafana/grafana-oss:4.3	3000	influxdb	grafana-storage + provisioning.ro
sensors	build: ./sensors/	-	broker, nodered	simulated_env:/simulated_env
actuators	build: ./actuators/	-	broker, nodered	simulated_env:/simulated_env
manager_app	build: ./managerApplication/	8089	-	simulated_env:/simulated_env
Network: net-test (bridge) Configuration: All services share .env file Deploy: docker compose up -d --build				

Figure 14: Docker configuration summary table.